

3.3 可视化模块（C 负责）

可视化模块负责将算法模块输出的图结构以直观、可交互的方式呈现在画布上，并支持关系图的图片导出。该模块由 GraphPanel、LayoutManager 和 ImageExporter 三个核心类组成，分别承担关系图的绘制与交互、节点坐标计算、图片导出封装三项职责。可视化模块位于 view 包和 io 包中，对外通过 setGraph、setHighlightedCycle、setSelectedOrder、switchLayout、resetView、exportPNG 等接口与 GUI 模块、算法模块协作，其内部实现细节不暴露给其他模块。

3.3.1 类设计

可视化模块的三个核心类职责划分如下：GraphPanel 负责关系图的绘制与用户交互，LayoutManager 负责根据图结构计算节点坐标，ImageExporter 负责图片导出的目录选择与文件命名封装。三个类之间的调用关系为：GraphPanel 持有 LayoutManager 实例用于布局计算，ImageExporter 通过调用 GraphPanel.exportPNG 完成实际导出。

GraphPanel 继承自 JPanel，是可视化模块的核心组件，其主要字段与方法如表 3-4 所示。字段部分保存当前图数据、三种高亮状态、视图变换参数以及节点坐标映射；方法部分包括对外接口、交互处理、绘制方法和工具方法四类。

表 3-3 GraphPanel 类的主要字段与方法

类型	名称	职责
字段	graph	当前图数据，由 setGraph 注入
字段	highlightedCycle / selectedOrder / selectedNode	三种高亮状态：环路径、拓扑序、选中节点
字段	nodePositions	节点名到坐标的映射，由 LayoutManager 计算后写入
字段	scale / offsetX / offsetY	当前视图的缩放比例与平移偏移量
方法	setGraph(Graph) / setHighlightedCycle(List) / setSelectedOrder(List[,int]) / setSelectedNode(String)	对外接口，供 GUI 模块调用
方法	switchLayout(int) / resetView() / getScale() / zoomIn() / zoomOut()	布局切换与视图控制接口

方法	<code>exportPNG(File)</code>	导出完整关系图为 PNG
方法	<code>paintComponent(Graphics)</code>	Swing 绘制入口
方法	<code>drawEdges / drawNodes / drawLegend / drawZoomIndicator</code>	分别绘制边、节点、图例、缩放指示器
方法	<code>estimateNodeWidth(String)</code>	静态方法，根据文本宽度估算节点宽度

`LayoutManager` 负责根据图结构计算节点坐标，采用策略模式封装分层布局与环形布局两种算法，通过 `mode` 字段区分当前布局模式。其对外方法包括 `setMode(int)`、`getMode()` 和 `computeLayout(Graph, int, int, int)`，其中 `computeLayout` 返回节点名到坐标的映射表。

`ImageExporter` 是一个工具类，全部方法为静态方法。其核心方法 `chooseAndExport(Component, GraphPanel)` 弹出目录选择框并调用 `GraphPanel.exportPNG`，内部通过 `exportWithTimestamp` 生成带时间戳的文件名，同秒内重复导出自动加序号，避免覆盖。

三个核心类之间的调用关系如图 3-4 所示。`GraphPanel` 作为核心组件，内部持有 `LayoutManager` 实例用于布局计算；`ImageExporter` 则通过调用 `GraphPanel` 的 `exportPNG` 方法完成实际的图片输出。



图 3-5 可视化模块类调用关系

3.3.2 核心流程

3.3.2.1 图渲染流程

GraphPanel 继承自 Swing 的 **JPanel**，重写 **paintComponent** 方法作为绘制的统一入口。绘制流程分为三个步骤：视图变换、主体绘制、叠加层绘制。

(1) 视图变换。在绘制主体之前，先根据当前的 **scale**、**offsetX**、**offsetY** 对 **Graphics2D** 对象进行平移和缩放，使绘制内容与用户当前的缩放平移状态一致。

(2) 主体绘制。调用 **drawGraph** 方法，其内部先通过 **recomputeLayoutIfNeeded** 判断布局是否需要重新计算（当图变化、布局模式变化或画布尺寸变化时），再依次调用 **drawEdges** 和 **drawNodes** 绘制边和节点。

(3) 叠加层绘制。主体绘制完成后，单独创建一个 **Graphics2D** 对象（不套用视图变换），在画布右上角绘制图例，在左下角绘制缩放比

例指示器，保证这两个元素始终固定在画布位置，不随缩放平移移动。

整个绘制流程从 `paintComponent` 入口开始，依次经过视图变换、主体绘制和叠加层绘制三个阶段，具体流程如图 3-5 所示。

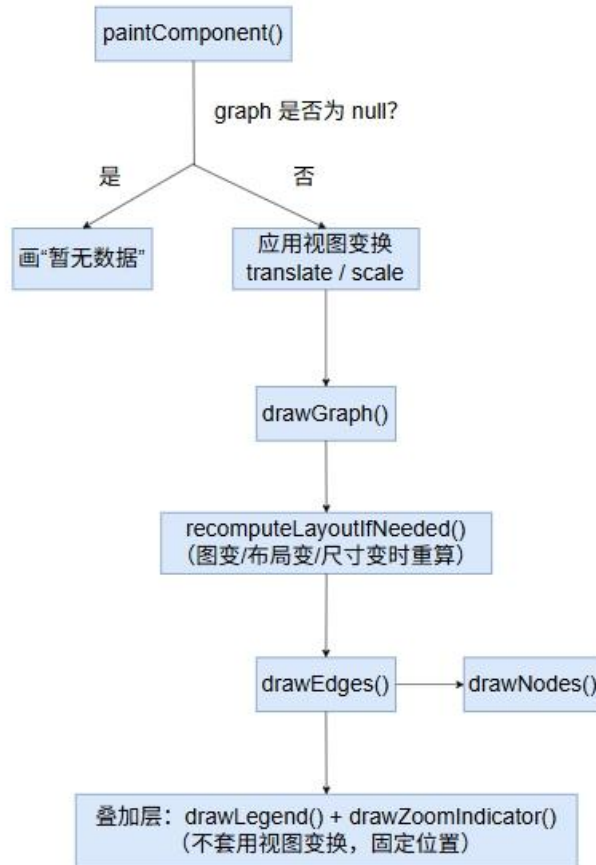


图 3-6 GraphPanel I 绘制流程

3.3.2.2 交互流程

可视化的交互包括缩放、平移、节点拖动、单击高亮、双击查看信息五类。

(1) 缩放。实现两种方式。滚轮缩放通过 `MouseWheelListener` 监听鼠标滚轮事件，以鼠标位置为中心调整缩放比例，缩放范围限制在 0.3 至 3.0 倍之间。按钮缩放通过 `zoomIn()` 和 `zoomOut()` 方法以画布中心为基准调整缩放。两种方式共用 `adjustScale` 内部方法统一处理缩放和平移偏移量的同步计算。

(2) 平移。监听鼠标按下、拖动、释放事件。当鼠标按下空白处时记录起始屏幕坐标，拖动过程中累加偏移量到 `offsetX` 和 `offsetY`，实现画布平移。

(3) 节点拖动。当鼠标按下命中节点时，记录该节点和起始模型坐标，拖动过程中更新该节点在 `nodePositions` 中的坐标，实现节点位置的微调。

(4) 单击高亮。单击节点时将该节点名存入 `selectedNode` 字段，触发重绘，该节点以黄色高亮显示；单击空白处取消选中。

(5) 双击查看信息。双击节点时通过 `hitNode` 方法进行命中检测，弹出对话框显示该节点的入度、出度、先修课程和后继课程。

3.3.2.3 PNG 导出流程

`exportPNG` 方法的执行流程分为四步。

(1) 确保布局已计算。调用 `recomputeLayoutIfNeeded` 保证 `nodePositions` 中保存的是当前图的最新坐标。

(2) 计算包围盒。遍历所有节点位置，根据节点宽度、自环高度和边距，计算所有节点的最小和最大横纵坐标，得到能容纳全部节点的最小包围盒。

(3) 创建图片并绘制。图片高度为节点包围盒高度加上图例高度和图例间距。创建 `BufferedImage` 对象后，将坐标原点平移到包围盒左上角，并向下偏移图例高度，绘制所有边和节点。此步骤不套用当前的缩放和平移，保证导出的是 1:1 的完整关系图。最后将图例绘制到右上角的预留空间。

(4) 写出文件。通过 `ImageIO.write` 将图片写入指定文件。

3.3.3 核心设计

3.3.3.1 分层布局算法

分层布局的目标是让关系图层次分明、减少边交叉，算法分为三个步骤。

第一步，计算每个节点的拓扑层号。使用 Kahn 算法的变体：初始入度为零的节点在第 0 层，逐层递推；如果图中存在环，环上的节点无法被处理，统一放到最末层兜底，保证布局仍然可读。

第二步，同层节点排序。采用重心法：每个节点取其所有前驱节点横坐标的平均值作为排序键，使得节点尽量靠近它的前驱，从而显著减少边的交叉。对于无前驱的节点（第 0 层），按名称字典序排序。

第三步，分配坐标。纵坐标按层号均匀分布，并保证相邻层之间有最小间距。横坐标根据每个节点的实际宽度加上最小间距，在画布宽度内均匀展开。节点宽度由 `estimateNodeWidth` 方法动态估算，中文字符按 14 像素、西文字符按 8 像素累加，保证课程名不溢出。

分层布局的伪代码描述如下。

输入: Graph graph, 画布宽 w, 高 h, 节点半径 r

输出: 每个节点的 (x, y) 坐标

layerMap $\leftarrow$ computeLayer(graph) // 步骤 1: 计算层号

byLayer $\leftarrow$ 按层号分组(layerMap) // 步骤 2: 分组

for each layer lv:

 同层节点按重心法排序

 计算该层所有节点的总宽度 + 最小间距

 在画布宽度内均匀分配横向位置

 纵坐标 = 层号 $\times$ 层间距

返回坐标映射

3.3.3.2 环形布局算法

环形布局作为含环图或特殊场景的兜底方案, 将节点均匀分布在圆周上。设画布宽为 w、高为 h、节点半径为 r, 则圆心为 (w/2, h/2), 半径为 $\min(w, h)/2 - r - 40$, 第 i 个节点的角度为 $2\pi i/n - \pi/2$ 。节点的横坐标为 圆心 x + 半径 $\times \cos(\text{角度})$, 纵坐标为 圆心 y + 半径 $\times \sin(\text{角度})$ 。环形布局的计算复杂度为 $O(n)$, 节点数增加时圆周上的节点密度增大, 但不会出现重叠。

为直观对比两种布局算法的视觉效果, 使用同一份图 1 数据分别应用分层布局 and 环形布局, 结果如图 3-6 所示。分层布局突出课程之间的先修层级关系, 环形布局则将所有节点均匀分布在圆周上, 适合含环图或需要整体观察节点关系的场景。

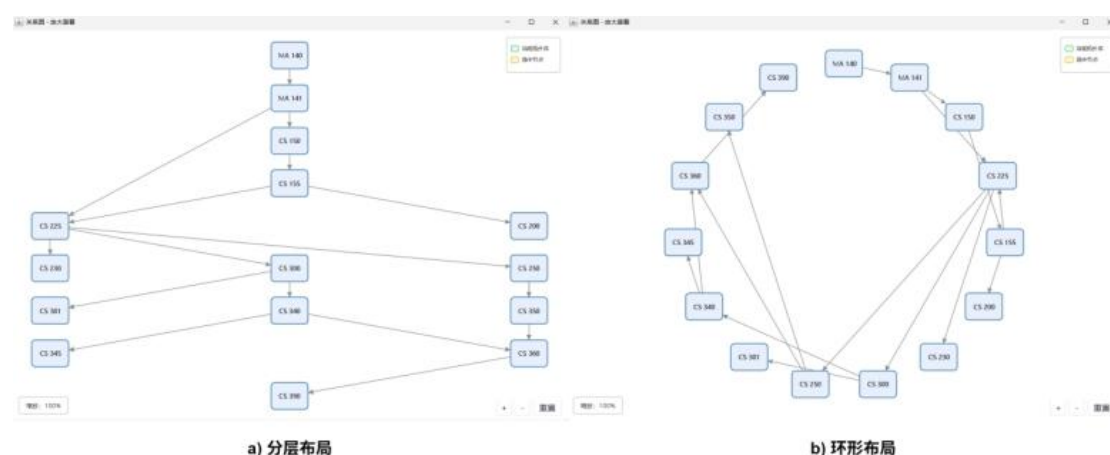


图 3-7 分层布局与环形布局效果对比

3.3.3.3 高亮与序号标注

可视化模块支持三种高亮状态, 在 drawNodes 方法中按优先级叠加。

(1) 环路径高亮。当 highlightedCycle 列表非空时, 环路径上的节点使用红色填充和红色边框, 同时环上的边在 drawEdges 中加粗并变红, 帮助用户定位循环依赖。

(2) 拓扑序高亮。当用户单击结果列表某条序列时，selectedOrder 被设置，同时传入该序列在结果列表中的序号作为颜色索引。可视化模块采用 5 色轮换色板（绿、橙、蓝、紫、红），根据颜色索引取对应颜色绘制节点和属于该拓扑序的边。同时，每个节点右上角绘制带数字的小圆圈徽章，显示该节点在当前拓扑序中的位置（1、2、3……），使不同拓扑序列的顺序差异一目了然。

(3) 选中节点高亮。当用户单击某个节点时，该节点使用黄色填充和橙色粗边框高亮显示。

三种高亮状态按顺序判断，选中节点优先级最高，环路径和拓扑序次之。

当用户在结果列表中单击不同序列时，画布上的节点会按照该序列的顺序着色并显示序号徽章，不同序列的效果对比如图 3-7 所示。图中左侧为第 1 条序列的高亮效果，右侧为第 2 条序列的效果，可以看出节点颜色和序号会随着选中序列的变化而更新。

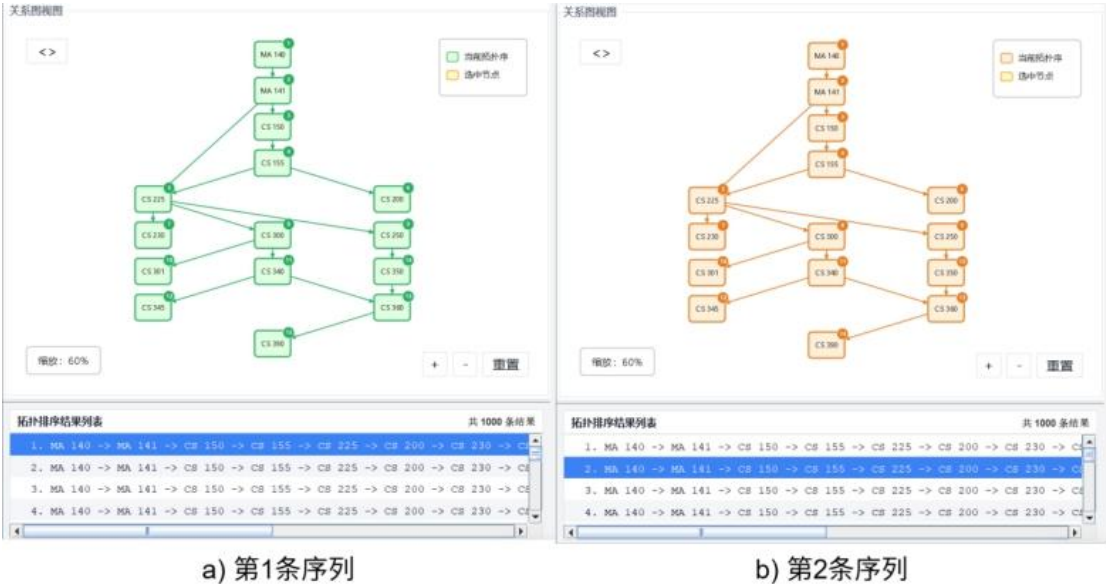


图 3-8 拓扑序高亮与序号标注效果

3.3.3.4 完整图导出

PNG 导出是可视化模块对外的重要功能。最初版本直接按面板尺寸创建图片并套用当前的缩放和平移，在测试中发现两个问题。

问题一：平移或缩放画布后导出，图片会变成空白或只剩图例，因为节点坐标被视图变换推到了图片边界之外。修复方法是：忽略当前的缩放和平移，遍历所有节点计算包围盒，按原始比例完整绘制，保证无论用户如何缩放或平移，导出的始终是完整关系图。

问题二：导出完整图后，右上角图例会遮挡正好位于该位置的节点或自环。修复方法是：在计算图片高度时为图例预留独立空间（图例高度 68 像素

+ 间距 20 像素)，并将节点整体向下平移这段距离，彻底避免图例与节点重叠。

此外，ImageExporter 封装了导出流程，弹出目录选择框，由用户指定保存路径和文件名，默认文件名为 graph.png；导出成功时弹出中文提示，导出失败时给出中文错误提示。

3.3.3.5 放大查看窗口

为了便于观察大规模关系图，可视化模块提供了放大查看功能。GraphPanel 左上角有一个“放大查看”按钮，点击后弹出一个独立窗口，内部创建一个新的 GraphPanel 实例，复制主面板的图数据、高亮状态、选中节点和布局模式。为保证主面板和弹窗的布局一致，GraphPanel 新增 getLayoutMode() 方法，弹窗创建后调用 popout.switchLayout(this.getLayoutMode()) 同步布局模式，避免主窗口切换为环形布局后弹窗仍显示分层布局的问题。

3.3.3.6 算法复杂度

设节点数为 n ，边数为 m 。分层布局中，Kahn 变体计算层号的时间为 $O(n + m)$ ，重心法排序在最坏情况下需要为每个节点扫描所有其他节点以确定前驱，因此排序部分的复杂度为 $O(n^2)$ 。对于本项目的课程数据规模（ $n \leq 100$ ），该复杂度完全可以接受；若要扩展到更大规模，可通过维护邻接表反向索引将排序优化到 $O(n + m)$ 。环形布局的时间复杂度为 $O(n)$ 。导出流程中，遍历节点计算包围盒的时间为 $O(n)$ ，绘制边和节点的时间为 $O(n + m)$ ，因此整体复杂度为 $O(n + m)$ ，主要瓶颈在于布局计算而非导出本身。